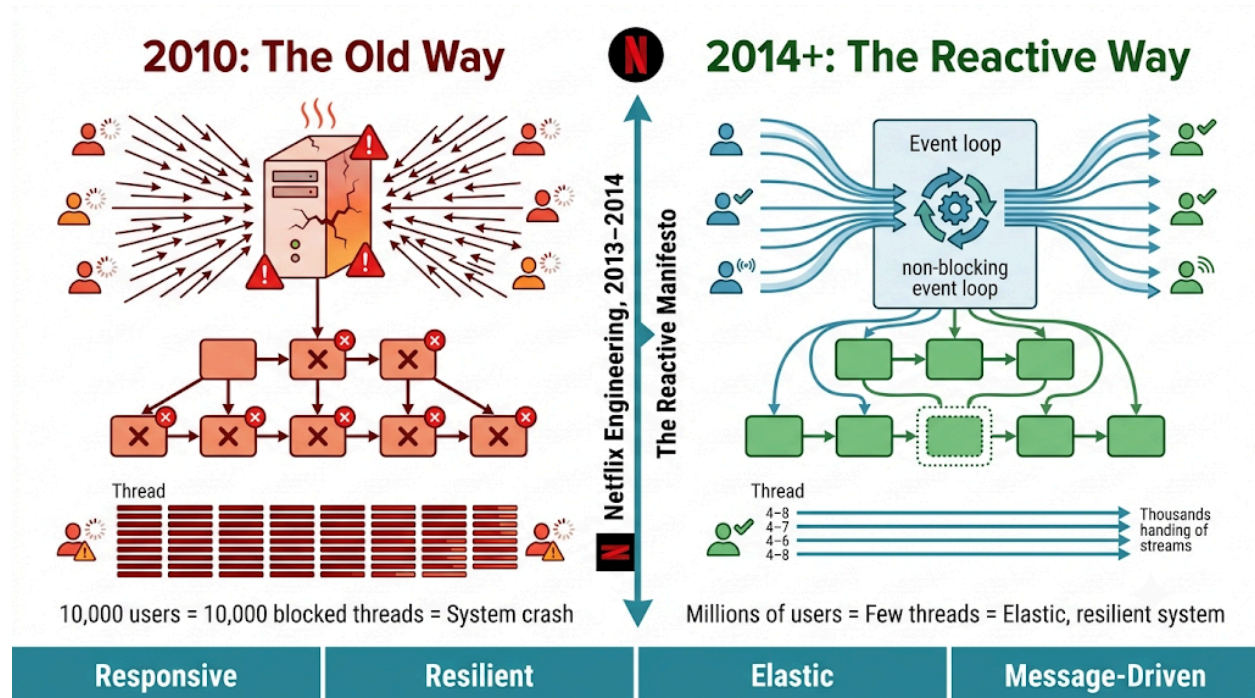
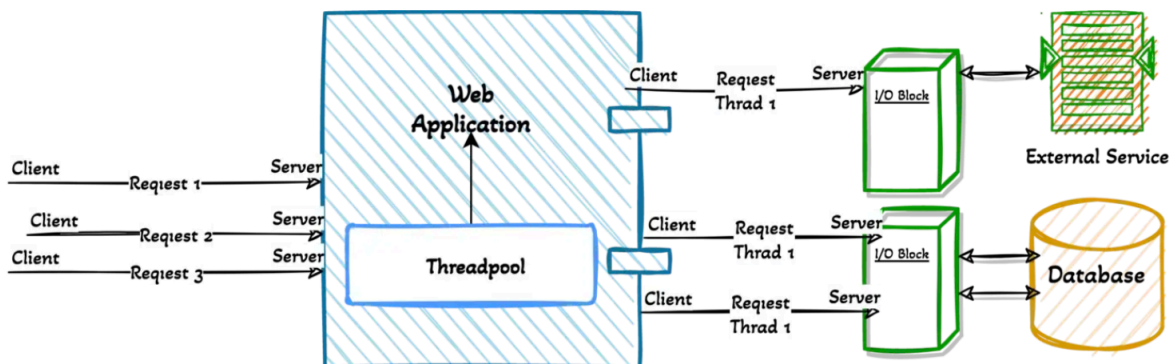


Introduction To Reactive Programming



Limitations of Traditional Blocking Apps



- One thread per request

Allocating one thread per request also limits the number of incoming requests. The system can only handle as many requests as the size of the thread pool.

- **Poor utilization of system resources**

In synchronous and blocking mode, to handle a certain number of requests in high traffic, new instances are required. This is a costly operation in terms of resource usage.

- No Flow Control. Fast Producer, Slow Consumer
- No way to handle a continuous stream of results

The Reactive Manifesto:

The Reactive Manifesto (2013) is not a library or a specification. It's a document that says: modern systems need to be built differently.

The Reactive Manifesto defines four essential properties of reactive systems. It defines the basic principles of reactive programming.

1. **Responsive:** The system **responds quickly** to user requests, even under varying load.
2. **Resilient:** The system **stays responsive** even when facing failures (errors, crashes, network issues).
3. **Elastic:** The system can **scale up and down** automatically based on load, efficiently using resources.
4. **Message-Driven:** The system uses **asynchronous message passing** between components. This enables loose coupling, backpressure, and resilience.



Netflix, Twitter (X), LinkedIn, and many banking systems follow the Reactive Manifesto. **Spring WebFlux + Project Reactor** is one of the best implementations of these principles in the Java world.

Reactive Streams Specification

Now, when the industry agreed on a goal (*Manifesto*), the next step was agreeing on a technical contract so that different libraries could interoperate. That's the

Reactive Streams Specification.

It consists of **4 key interfaces**:

1. Publisher

- Produces (emits) data.
- Responsible for sending items to the Subscriber.
- Has one method: `subscribe(Subscriber)`. It doesn't push data until a subscriber asks for it.

3. Subscription

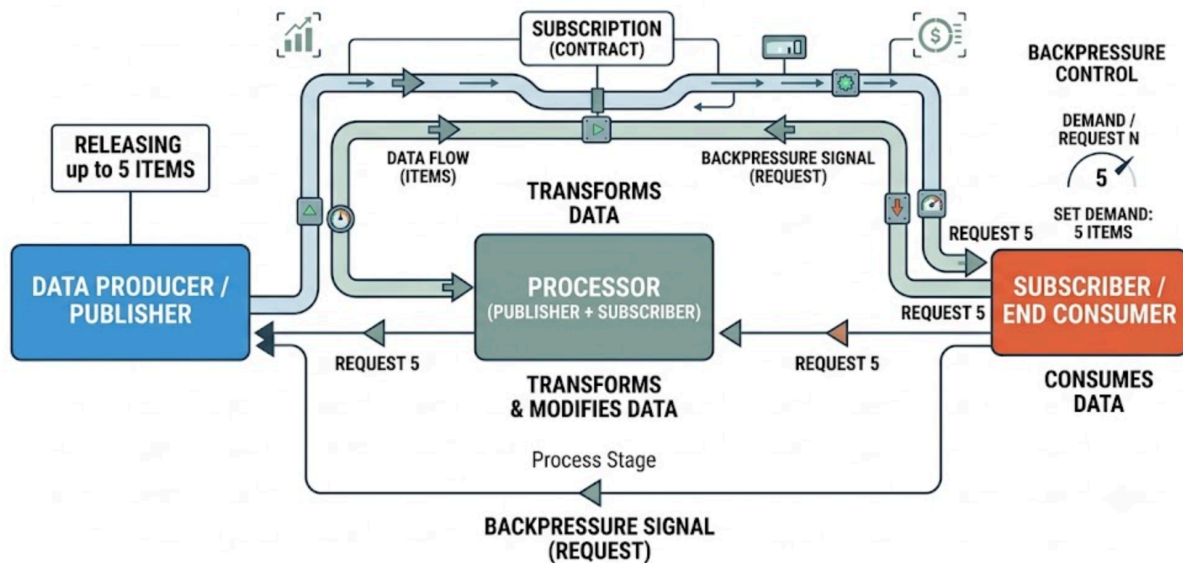
- The connection between Publisher and Subscriber. Used for **backpressure** control.
- The control channel between the two. The subscriber holds this and calls `request(n)` to ask for `n` more items, or `cancel()` to stop entirely. This is where backpressure lives.

2. Subscriber

- Consumes (receives) the data.
- Reacts to incoming items.
- Gets four callbacks: `onSubscribe`, `onNext`, `onError`, `onComplete`. You never call these yourself; the Publisher calls them.

4. Processor

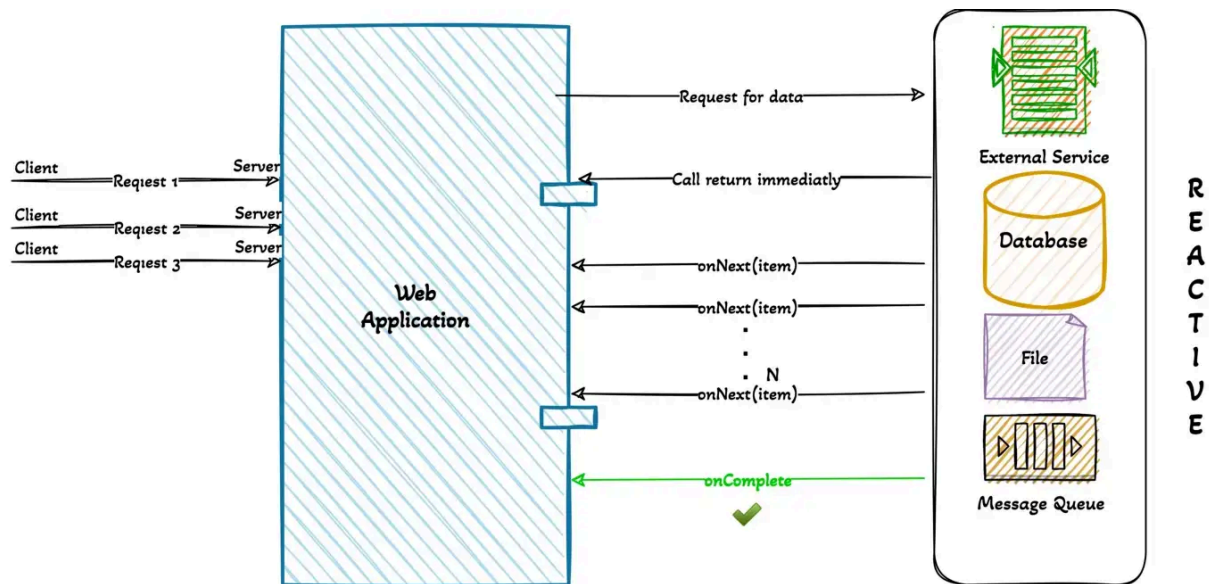
- Combination of Publisher + Subscriber.
- Can transform data while passing it forward.
- It sits in the middle of a pipeline, transforming items. In practice, you rarely implement this directly. You use operators like `map` and `flatMap` instead.



All major reactive libraries in Java implement this specification. Here are the most commonly used implementations:

Library	Main Use Case	Key Features	Status
Project Reactor	General purpose, Spring integration	Flux & Mono, excellent performance	Most popular
RxJava	Android, general reactive	Rich operators, very mature	Very popular
Vert.x	Reactive microservices	Polyglot, event-driven	Good for async
Java Flow API	Built-in (JDK 9+)	Simple, no extra dependency	Standard

How does Reactive Programming work?



Steps by Step Process:

1. Create Data Streams

Turn everything into observable event streams: user actions, HTTP requests, database queries, variable changes, messages, etc.

2. Emit Events

For each result from a data source (DB, API, file), create an `onNext` event with the data item.

3. Handle Completion

When the data source finishes successfully, trigger an `onComplete` event.

4. Handle Errors

If an error occurs, trigger an `onError` event.

5. React to Changes

Observers continuously watch the stream and automatically respond to `onNext`, `onComplete`, or `onError` events.

🕵️ Sneak-peak

```
// Thread is BLOCKED here
User user = userRepo
    .findById(id);    //
```

```
// Thread is FREE during
I/O
return userRepo
```

```

waits...

Order order = orderRepo
    .findByUser(user); //
waits...

return order;        // ~350ms
idle

```

```

    .findById(id)
    .flatMap(user ->
        orderRepo.findByIdUs
er(user)
    ); // pipeline, non-b
locking

```

What is Reactive Programming?

Reactive programming is a new paradigm that was introduced recently and it has fast become a very popular topic among software engineers.

- **Declarative Paradigm**

You declare *what* should happen when data arrives, not *how* to wait for it. The framework handles the rest.

- **Asynchronous Data Streams**

Everything is a stream → HTTP responses, DB results, events, clicks. Data flows through pipelines of operators.

- **Propagation of Change**

When an upstream value changes, dependent computations automatically re-execute, like a spreadsheet formula.

- **Backpressure support**

Backpressure and error operators prevent cascading failures. Backpressure support is available in data streams. It offers predictable response times due to its asynchronous and non-blocking structure.

- **Non-blocking I/O**

Threads never idle-wait for a DB call or HTTP request. They are freed and reused for other work immediately. It offers better use of system resources. You can handle 10× more concurrent users with the same number of threads.

When Should You Go Reactive?

- **High-Concurrency APIs**

Notification services, chat systems, and real-time feeds handling 100K+ simultaneous connections, e.g. Twitter (real-time interactions), LinkedIn (messaging and feeds)

- **Streaming Data**

Live dashboards, stock tickers, IoT sensor streams, and data flowing continuously to clients. e.g. Netflix (video streaming), Goldman Sachs (live stock price tickers)

- **Heavy I/O Workloads**

Applications whose bottleneck is DB / network I/O, not CPU. e.g Amazon, Alibaba (handle massive requests)